

A Practical but Lighter Introduction to gemstone-rs

Rust, GemStone/S, and the sharp edges you actually need to know about



Generated from the Markdown sources in gemstone-rs/docs.

Contents

A Practical but Lighter Introduction to gemstone-rs

Part 1: Why gemstone-rs Exists

The First Promise

The Second Promise

The First Useful Command

Part 2: Sessions and Transactions

The Transaction Wrapper

Manual Control

Run the Example

Part 3: OOPs, Values, and the Browser

Values

Retaining OOPs

Browser API

Part 4: Codegen, Explorer, and VS Code

Codegen Is a Contract, Not a Shortcut

Discover From a Live Stone

Use the Explorer for Fast Feedback

Use VS Code When You Want a Guided Workflow

The Practical Finish Line

A Practical but Lighter Introduction to gemstone-rs

This is the guided version of the docs. It still shows real APIs, real command lines, and real failure modes, but it lets the explanation breathe a little.

Read in order:

1. [Why gemstone-rs Exists](#)
2. [Sessions and Transactions](#)
3. [OOps, Values, and the Browser](#)
4. [Codegen, Explorer, and VS Code](#)

The short version:

```
cargo add gemstone-rs
cargo install gemstone-rs-cli
gemstone-rs eval "3 + 4"
```

If that prints `7`, Rust and GemStone/S have successfully shaken hands.

Part 1: Why gemstone-rs Exists

GemStone/S is not a SQL database wearing a new hat. It stores objects, runs Smalltalk, and lets sessions send messages to live objects. Rust is not a dynamic scripting language. It asks you to be clear about ownership, errors, and resources.

That sounds like a mismatch until you notice the useful part: GemStone wants a client that respects session boundaries, transactions, and object identity. Rust wants APIs that make those boundaries explicit. `gemstone-rs` exists at that meeting point.

The First Promise

The first promise is small:

```
use gemstone_rs::{Config, Session, Value};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env())?;
```

```
assert_eq!(session.eval("3 + 4")?, Value::SmallInt(7));
Ok(())
}
```

There is no web server hidden in the middle. There is no Python process acting as a translator. The Rust process loads GemStone's GCI library and talks to the stone directly.

The Second Promise

The second promise is separation:

Crate	Job
<code>gemstone-gci</code>	Unsafe dynamic GCI loading and raw ABI calls.
<code>gemstone-rs</code>	Safe Rust API.
<code>gemstone-rs-cli</code>	Command-line workflows.
<code>gemstone-rs-explorer</code>	Local HTTP browser and codegen proving ground.

The unsafe part is boxed into the low-level crate. The public API talks in terms of `Config`, `Session`, `Op`, `Value`, and `Result`.

The First Useful Command

```
gemstone-rs eval "3 + 4"
```

If this works, you know the process can:

- load `libgcirpc`
- read credentials
- log in
- evaluate Smalltalk
- marshal a simple result

That is a lot of proof for one tiny command.

Part 2: Sessions and Transactions

A GemStone session is not just a socket with better manners. It is logged in, transactional, and attached to server-side state. Treat it casually and your code will eventually ask why a write "worked" and then vanished.

`gemstone-rs` makes the session visible:

```
let mut session = Session::login(Config::from_env())?;
println!("session id: {}", session.session_id());
session.logout()?;
```

`Session` also logs out on drop. Explicit `logout()` is still useful when a command-line example wants to show the full lifecycle.

The Transaction Wrapper

```
session.transaction(|session| {
    let value = session.new_string("committed");
    session.global_put("GemStoneRsCommitted", value)
})?;
```

This wrapper is intentionally boring:

- `Ok` means commit
- `Err` means abort
- the error is returned to the caller

Boring transaction helpers are good. Exciting transaction helpers are usually a future incident report.

Manual Control

Sometimes you want the raw controls:

```
let needs_commit = session.needs_commit()?;
let in_transaction = session.in_transaction()?;
session.commit()?;
session.abort()?;
```

Manual control is useful for maintenance tools and diagnostics. Application code should usually prefer the transaction wrapper.

Run the Example

```
cargo run -p gemstone-rs --example transactions
```

It writes one committed value and then intentionally takes the abort path for a second value. The point is not drama. The point is knowing which writes survive.

Part 3: OOPs, Values, and the Browser

GemStone object identity is represented by OOPs. Rust programmers are already comfortable with explicit handles, so `gemstone-rs` does not hide them.

```
let seven = session.smallint_oop(7);
let printed = session.perform_oop(seven, "printString", &[])?;
println!("{}", session.fetch_string(printed)?);
```

If you want a marshalled result instead, use `perform`:

```
let value = session.perform(seven, "class", &[])?;
println!("{value:?}");
```

Values

Simple results become `Value`:

```
use gemstone_rs::Value;

match session.eval("3 + 4")? {
    Value::SmallInt(value) => println!("{value}"),
    other => println!("{other:?}"),
}
```

Everything else can stay as an `Oop`. That is useful because not every GemStone object should pretend to be a Rust value.

Retaining OOPs

When a raw OOP must live across calls, retain it:

```
let text = session.new_string("retained"?);
let handle = session.retain_oop(text)?;
println!("{}", handle.oop().raw());
```

The handle releases on drop. Your future self will appreciate that.

Browser API

The browser API lets Rust inspect code in the stone:

```
use gemstone_rs::browser::{Browser, ALL_PROTOCOLS};

let mut browser = Browser::new(&mut session);
let dictionaries = browser.dictionaries()?;
let methods = browser.methods("Object", ALL_PROTOCOLS, false, "")?;
let source = browser.source("Object", "printString", false, "")?;
```

The same browser API powers:

- `gemstone-rs browse ...`
- `gemstone-rs-explorer`
- the VS Code sidebar

That is the pattern for the project: one real Rust API, multiple tools on top.

Part 4: Codegen, Explorer, and VS Code

The first three steps got Rust connected to GemStone/S, showed how values move across the boundary, and gave you browser calls for dictionaries, classes, protocols, methods, and source. That is enough to build tools. It is not yet the pleasant path for an application team that wants checked-in Rust wrappers and a repeatable workflow.

This is where codegen, the local explorer, and the VS Code workbench become useful.

Codegen Is a Contract, Not a Shortcut

The goal of gemstone-rs codegen is not to hide GemStone/S. The goal is to turn a known set of GemStone classes and methods into reviewed Rust source files.

Start with a config:

```
output = src/generated/gemstone_wrappers.rs
class = Object
method = Object>>printString | return=String | doc=Return the receiver printString.
method = Object>>class | return=Oop | doc=Return the receiver class.
```

Preview first:

```
cargo run -p gemstone-rs-cli -- codegen preview examples/codegen/gemstone-rs.codegen
```

Check before committing:

```
cargo run -p gemstone-rs-cli -- codegen check
```

Generate when the preview is what you want:

```
cargo run -p gemstone-rs-cli -- codegen generate examples/codegen/gemstone-rs.codegen
```

The useful habit is simple: preview, diff, check, generate, then review the generated file like any other source file.

Discover From a Live Stone

When you are connected to a live stone, the CLI can create a starting config:

```
cargo run -p gemstone-rs-cli -- codegen discover examples/codegen/object.codegen
Object
```

That gives you a concrete list to prune. For a real project, the codegen config should be small and deliberate. Start with the classes your Rust code actually calls. Add more only when the application needs them.

Use the Explorer for Fast Feedback

The explorer is a local HTTP app that sits on top of the same Rust API:


```
cargo run -p gemstone-rs-explorer -- --port 8787
```

Open these endpoints while developing:

```
http://127.0.0.1:8787/  
http://127.0.0.1:8787/api/status  
http://127.0.0.1:8787/api/browse/dictionaries  
http://127.0.0.1:8787/api/browse/classes?dictionary=UserGlobals  
http://127.0.0.1:8787/api/browse/methods?class=Object&protocol=--%20all%20--  
http://127.0.0.1:8787/api/codegen/preview?config=examples/codegen/gemstone-rs.codegen  
http://127.0.0.1:8787/api/codegen/diff?config=examples/codegen/gemstone-rs.codegen  
http://127.0.0.1:8787/api/codegen/check?config=examples/codegen/gemstone-rs.codegen
```

Keep it bound to `127.0.0.1`. Treat eval and write operations as explicit local developer actions.

Use VS Code When You Want a Guided Workflow

The VS Code workbench is intentionally a thin layer over the Rust CLI and local explorer. It should not become a second GemStone client. It should make the common commands easier to discover.

Useful commands:

```
GemStone RS: Configure Workbench  
GemStone RS: Verify Workbench Setup  
GemStone RS: Launch Explorer  
GemStone RS: Run Codegen Check  
GemStone RS: Preview Codegen Wrappers  
GemStone RS: Generate Codegen Wrappers  
GemStone RS: Open Codegen Docs
```

The intended codegen loop in VS Code is:

1. Configure the CLI path, checkout path, explorer path, and GemStone environment.
2. Verify the setup report.
3. Browse the stone.
4. Preview generated wrappers.
5. Review the diff.
6. Generate wrappers.
7. Run the Rust tests.

The Practical Finish Line

The first useful milestone is not a huge generated API. It is a tiny, tested wrapper that proves the path:

```
let object = session.resolve("Object"?;
let text = session.perform_string(object, "printString", &[])?;
assert(!text.is_empty());
```

Once that works, the rest of the workflow is mostly scale: more classes, more methods, better names, better docs, and stronger tests.

This PDF was generated locally from docs/. If you edit the Markdown sources, rerun python docs/build_pdf_docs.py.